

Mininet 大作业

项目概况：

在本项目中，您将了解 SDN（Software-Defined Networking）。使用 Virtualbox、Mininet 和 Pox 作为 OpenFlow 协议的实现者，使用 SDN 原语构建简单的网络。使用 Virtualbox、Mininet 和 Pox 作为 OpenFlow 协议的实现者，您将使用 SDN 原语构建一些简单的网络。

- 首先，您将学习使用 [mininet](#)，一个支持 SDN 的网络模拟器。
- 对于第二部分，您将使用 POX 通过 SDN 控制器确定的 OpenFlow 规则实现简单的 L2 静态防火墙。
- 对于第三部分，您将构建一个完整的网络，其中多个交换机能够处理静态拓扑中的流量。
- 在第四部分中，您将修改第 3 部分解决方案以实现一个第 3 层学习型 IP 路由器，该路由器能够处理 ARP，并在子网之间动态路由流量。

背景介绍：

SDN 和 OpenFlow：

SDN（Software-Defined Networking）是最近提出的一种网络范例，其中数据平面和控制平面彼此分离。可以将控制平面视为网络的“大脑”，即它负责做出所有决策。例如如何转发数据，而数据平面才是实际移动数据的部分。在传统网络中，控制平面和数据平面紧密集成并实施在组成网络的转发设备中。SDN 控制平面由“控制器”实施，数据平面由“交换机”实施。控制器充当网络的“大脑”，并向交换机发送有关如何处理流量的命令（“规则”）。OpenFlow 已成为事实上的 SDN 标准，并指定控制器和交换机如何通信以及控制器在交换机上安装的规则。

Mininet：

Mininet 是一个软件堆栈，可在您的计算机/笔记本电脑上创建虚拟网络。它通过创建主机命名空间（h1、h2 等）并通过虚拟接口连接它们来完成此任务。因此，当我们在 Linux 命名空间 h1 和 h2 之间运行 ping 时，ping 将从 h1s 命名空间通过为 h1 和 h2 创建的虚拟接口对运行，然后到达 h2。如果 h1 和 h2 通过交换机连接，如 Mininet 演练中的 python 代码所示，则 ping 将传输多个虚拟接口对。我们将使用的交换机运行 OpenVSwitch（OVS），这是一种软件定义的网络堆栈。Mininet 将在交换机上的每个虚拟端口与每个连接的主机之间连接额外的虚拟接口。主机名空间允许每个主机看到相同的文件系统，但作为自己的进程运行，该进程将与其他每个主机进程分开运行。在 Ubuntu 映像上运行的 OVS 版本支持 OpenFlow。

Pox：

Pox 是 OpenVFlow 控制器的研究/学术实现。它提供 Python 钩子来编程基于 mininet 的软件模拟网络。

环境配置

本次实验需要重新创立一个虚拟环境，请按照流程重新创立虚拟机。不要用前五次实验的环

境!!!

1. 切换到 **mininet-environment** 的目录，运行 **vagrant up** 与 **vagrant ssh** 命令。利用 **ls** 命令查看目录，结果应该如下图所示。

```
vagrant@mnvm:~$ ls
mininet project-1 project-2
```

2. 在虚拟机中运行 **sudo PYTHON=python3 mininet/util/install.sh -a** 来下载 mininet 下载成功后的输出与目录如下：

```
make[1]: Nothing to be done for 'install'.
make[1]: Leaving directory '/home/vagrant/oflops/doc'
Enjoy Mininet!
```

```
vagrant@mnvm:~$ ls
mininet oflops oftest openflow pox project-1 project-2
```

安装成功了可以忽略下面的环境配置部分，直接阅读 **Part1-4**。

3. 如果出现下图的错误：

```
Setting up pkg-config (0.29.1-0ubuntu4) ...
Processing triggers for man-db (2.9.1-1) ...
Cloning into 'openflow'...
fatal: unable to access 'https://github.com/mininet/openflow/': Failed to connect to github.com port 443: Connection timed out
vagrant@mnvm:~$
```

请打开 **install.sh** 文件，注释掉报错的部分

```
vagrant@mnvm:~$ cd mininet
vagrant@mnvm:~/mininet$ cd util
vagrant@mnvm:~/mininet/util$ vim install.sh
```

```
# The following will cause a full OF install, covering:
# -user switch
# The instructions below are an abbreviated version from
# http://www.openflowswitch.org/wk/index.php/Debian_Install
function of {
    echo "Installing OpenFlow reference implementation..."
    cd $BUILD_DIR
    $install autoconf automake libtool make gcc
    if [ "$DIST" = "Fedora" -o "$DIST" = "RedHatEnterpriseServer" ]; then
        $install git pkgconfig glibc-devel
    elif [ "$DIST" = "SUSE LINUX" ]; then
        $install git pkgconfig glibc-devel
    else
        $install git-core autotools-dev pkg-config libc6-dev
    fi
    # was: git clone git://openflowswitch.org/openflow.git
    # Use our own fork on github for now:
    git clone https://github.com/mininet/openflow
    cd $BUILD_DIR/openflow

    # Patch controller to handle more than 16 switches
    patch -p1 < $MININET_DIR/mininet/util/openflow-patches/controller.patch

    # Resume the install:
    ./boot.sh
```

运行 **git clone <https://github.com/mininet/openflow>** 手动下载。

下载完成后的目录如下：

```
vagrant@mnvm:~$ ls
mininet openflow project-1 project-2
vagrant@mnvm:~$
```

再次运行更改过的 **install.sh**，**sudo PYTHON=python3 mininet/util/install.sh -a**

然而在实际运行 **install.sh** 时，可能会出现别的下载错误，可以仿造上面的例子，在 **install.sh** 中注释掉报错部分，手动下载报错的包，然后再次运行更改后的 **install.sh**。

Part1 : Mininet 入门

学习目标：

- 使用 Mininet（一种广泛用于研究和实验的自包含网络环境）构建测试网络拓扑。
- 使用 Mininet 内置的基本调试工具测试虚拟主机之间的连通性并获取其拓扑中 openflow 交换机的状态。

请 cd 到 mininet-environment 目录，然后运行 `vagrant up` 和 `vagrant ssh` 命令。（不要用前五次作业中的虚拟机，重新使用 `vagrant up` 创立一个

使用 Mininet:

[这里](#)描述了 mininet 的使用方法。你可以输入以下命令来运行它 `sudo mn`:

还有一个非常有用的 [Mininet 社区 wiki](#)，其中有大量关于如何有效使用 mininet 的最新信息。

在 mininet CLI 中，尝试运行其他命令，如 `help`、`net`、`nodes`、`links`、`dump`。Mininet 从您可以访问的默认网络启动。查找主机的 mac 地址、连接的以太网端口以及系统中的主机名。

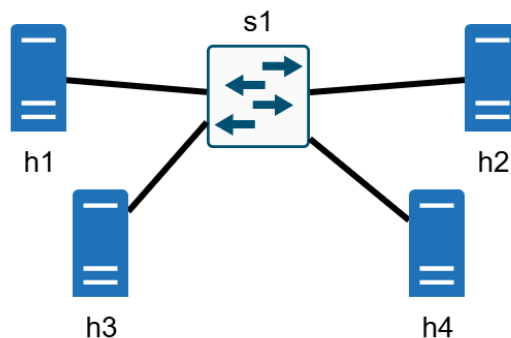
Mininet 拓扑编程:

Mininet 也可以使用 Python 编程语言进行编程。我们[在此处](#)提供了一些示例拓扑，这些拓扑应该已下载到 VM 中并解压到 `~/project-1`。

在此目录中，您将找到两个不同的目录：`topo` 和 `pox`。暂时忽略 `pox` 目录（它在第 2 部分中使用）。在 `topo` 文件夹中有各种 python 文件。它们每个都为以下每个项目部分定义了一个拓扑。使用 `sudo python3 project-1/topos/part1.py` 来运行 `project 1` 文件。它将让您使用 python 脚本中定义的网络拓扑进入 CLI。

任务:

- 1.修改 `part1.py` 来表示以下网络拓扑:



- 2.使用 `sudo python3 project-1/topos/part1.py` 来运行 `project 1` 文件
- 3.在 CLI 中分别运行 `pingall`，`iperf h1 h4`，`dump`，这三条命令。然后将输出截图保存，附在实验报告中。（退出 CLI 可以输入 `exit` 或 `CTRL+Z`）

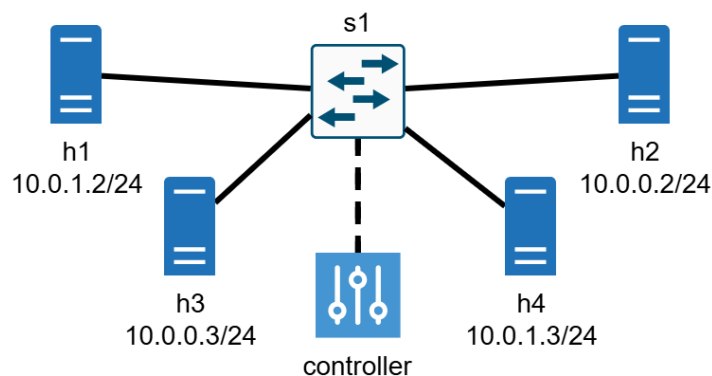
Part2: 使用 POX 的 SDN 控制器

学习目标:

- 运行自己的 POX openflow 控制器，向 mininet 拓扑中的交换机添加自定义规则。
- 构建一个能够通过单个可编程交换机将主机连接在一起的 OpenFlow SDN 控制器程序。
- 通过将数据包从交换机端口泛洪来建立连接。
- 创建匹配 -> 操作规则，根据第 2 层和第 3 层报头信息阻止或允许特定类型的流量通过交换机。

在第一部分中，我们使用 Mininet 的内部控制器进行了实验。在这一部分（以及以后的部分）中，我们将使用自己的控制器向交换机发送命令。我们将使用 Python 编写的 POX 控制器。

在本作业中，您将使用支持 OpenFlow 的交换机创建一个简单的防火墙。“防火墙”一词来源于建筑结构：防火墙是建筑中用来阻止火势蔓延的墙。就网络而言，它是一种通过不让指定流量通过防火墙来提供安全性的行为。这一功能有利于最大限度地减少攻击载体，限制暴露给攻击者的网络“表面”。在本部分中，我们将为您提供 Mininet 拓扑 `part2.py`，用于设置网络，其中假定远程控制器监听默认 IP 地址和端口号 127.0.0.1:6633。您无需（也不应）修改此文件。该脚本将设置的拓扑结构如下。请注意，h1 和 h4 位于同一子网，而 h2 和 h3 位于相同子网中。



对于第 2 部分，我们还将为您提供一个 POX 控制器的骨架代码： `a1part2controller.py`。您将在此文件中进行修改以创建防火墙。

`bootstrap` 应该设置从 `~/pox/pox/misc` 目录到 `~/project-1/pox/a1part2controller.py` 的链接。如果 `~/pox/pox/misc/a1part2controller.py` 上没有链接，您可以通过运行 `ln -s ~/project-1/pox/* ~/pox/pox/misc/` 来创建一个新链接。

然后，您可以使用命令 `sudo ~/pox/pox.py misc.a1part2controller` 来启动控制器。控制器运行后，您可以使用 `sudo python3 ~/project-1/topos/part2.py` 来运行 mininet。

Note: 使用终端多路复用器（如 `tmux` 或 `screen`）可以方便地在 VM 中的单独终端窗口中运行控制器。或者，您也可以打开多个 SSH 连接。这里推荐使用第二种方式，可以再打开 `git bash`，然后运行 `vagrant ssh`。将启动控制器的命令和运行 mininet 的命令分别在两个 ssh 连接中运行。

s1 需要实现的规则如下：

src ip	dst ip	protocol	Action
any ipv4	any ipv4	icmp	accept
any	any	arp	accept

any ipv4	any ipv4	-	drop
----------	----------	---	------

基本上，您的防火墙应该允许所有 ARP 和 ICMP 流量通过。但是，任何其他类型的流量都应被丢弃。将允许的流量泛洪到所有端口是可以接受的。要注意的是，流表根据最高优先级的规则进行匹配，优先级是根据规则在表中的顺序来确定的。当您在 POX 控制器中创建一个规则时，你还需要让 POX 将该规则“安装”到交换机中。这样，交换机会在几秒钟内“记住”该规则的处理方式。不要在控制器中单独处理每一个数据包！

Note: 要执行此操作，请查找 `ofp_flow_mod`。 [OpenFlow tutorial](#)（特别是“Sending OpenFlow messages with POX”）和 [POX Wiki](#) 都是了解如何使用 POX 的有用资源。

任务：

1. 运行 `ln -s ~/project-1/pox/* ~/pox/pox/misc/`
若报错，可通过 `ls -ld ~/pox/pox/misc/` 查看权限，通过 `sudo chmod 777 ~/pox/pox/misc/` 赋予权限。

```
vagrant@mnvm:~$ ls -ld ~/pox/pox/misc/
drwxr-xr-x 3 root root 4096 Dec  3 10:40 /home/vagrant/pox/pox/misc/

vagrant@mnvm:~$ ln -s ~/project-1/pox/* ~/pox/pox/misc/
ln: failed to create symbolic link '/home/vagrant/pox/pox/misc/a1part2controller.py': Permission denied

vagrant@mnvm:~$ sudo chmod 777 ~/pox/pox/misc/
vagrant@mnvm:~$ ln -s ~/project-1/pox/* ~/pox/pox/misc/
```

2. 修改 `a1part2controller.py` 文件，然后在多个 ssh 连接中，分别使用 `sudo ~/pox/pox.py misc.a1part2controller`，`sudo python3 ~/project-1/topos/part2.py` 来启动控制器和运行 mininet。请注意两条命令的先后顺序。
3. 如果遇到端口号被占用的情况

```
vagrant@mnvm:~$ sudo python3 project-1/topos/part1.py
Traceback (most recent call last):
  File "project-1/topos/part1.py", line 25, in <module>
    net = Mininet(topo=t)
  File "/usr/local/lib/python3.8/dist-packages/mininet/net.py", line 178, in __init__
    self.build()
  File "/usr/local/lib/python3.8/dist-packages/mininet/net.py", line 508, in build
    self.buildFromTopo( self.topo )
  File "/usr/local/lib/python3.8/dist-packages/mininet/net.py", line 475, in buildFromTopo
    self.addController( 'c%d' % i, cls )
  File "/usr/local/lib/python3.8/dist-packages/mininet/net.py", line 291, in addController
    controller_new = controller( name, **params )
  File "/usr/local/lib/python3.8/dist-packages/mininet/node.py", line 1593, in DefaultController
    return controller( name, **kwargs )
  File "/usr/local/lib/python3.8/dist-packages/mininet/node.py", line 1417, in __init__
    self.checkListening()
  File "/usr/local/lib/python3.8/dist-packages/mininet/node.py", line 1433, in checkListening
    raise Exception( "Please shut down the controller which is"
Exception: Please shut down the controller which is running on port 6653:
Active internet connections (servers and established)
```

方法 1: `sudo lsof -i:<port>` 查找占用端口的进程，`sudo kill -9 <pid>` 杀掉进程

方法 2: `sudo mn -c` 重启 mininet

4. 在进入的 CLI 中运行 `pingall`。请注意，h1 和 h4 应该能够互相 ping 通（h2 和 h3 也一样），但不能跨子网。此外，命令 `iperf h1 h4` 应该会失败（因为您阻止了 IP 流量）。这表现为命令挂起。
5. 运行 `dpctl dump-flows`，这应该包含您插入到交换机中的所有规则。请将 2，4 中的输出截图附在实验报告中。

在完成后面两部分前，请先采用以下命令运行 `project-2-starter bootstrap-p2.sh` 脚本。此

脚本将为 project2 创建类似于在 project 1 中创建的链接。

```
cd /home/vagrant/project-2
```

```
./bootstrap-p2.sh
```

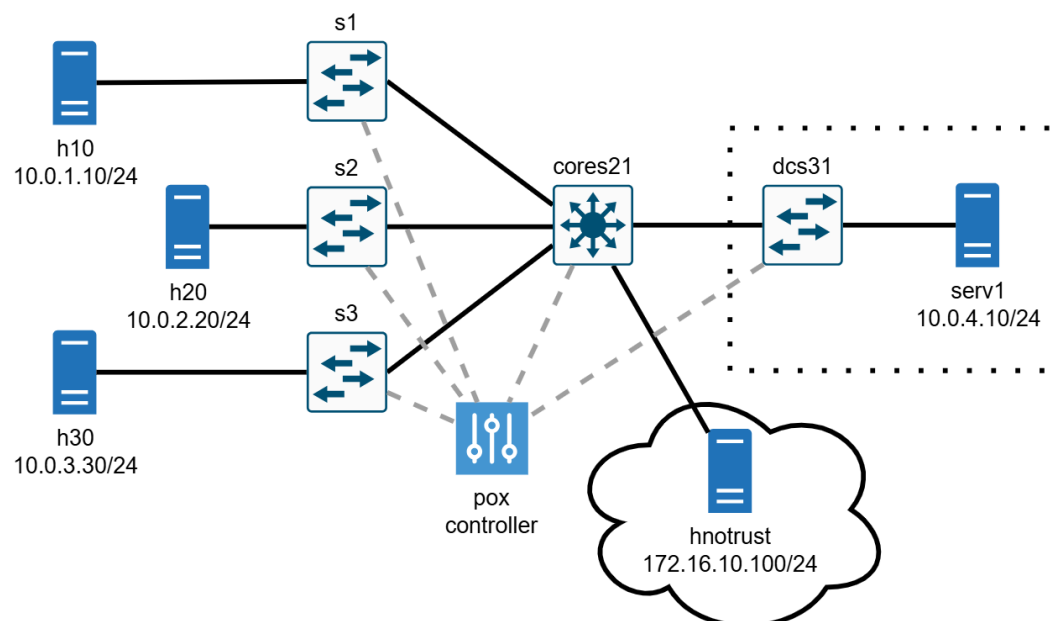
Part3: 更复杂的网络

学习目标:

- 根据网络中地址和拓扑的静态知识，创建匹配 -> 操作规则，以有效地将流量转发出特定端口，而不会造成泛洪。
- 创建匹配 -> 操作规则，根据静态寻址计划在多个不同的 L2 网络中正确转发 L3 流量。
- 创建一个 SDN 控制器程序，该程序能够配置多个交换机，并为不同类型的交换机指定不同的行为。

在 part2 中，您实现了一个简单的防火墙，允许 ICMP 数据包通过，但阻止所有其他数据包。对于这个项目，您将在此基础上进行扩展，以实现子网之间的 L3 路由，并为某些子网实现防火墙。这个想法是模拟一个更现实的网络。

我们将模拟一家小公司的网络。该公司有一栋三层楼，每层都有自己的交换机和子网。此外，我们在地下室的数据中心为所有服务器配备了一个交换机和子网，还有一个将所有设备连接在一起的核心交换机。请注意，名称和 IP 不可更改。与上一个作业一样，我们提供了拓扑 (project-2-starter/topos/part1.py) 以及骨架控制器 (project-2-starter/topos/a2part1controller.py)。与上 part2 一样，您只需修改控制器。



您的目标是允许流量在公司内所有授权主机之间传输。在这个作业中，你将被允许在次级路由器 (s1、s2、s3、dcs31) 上进行流量泛洪，方法与在 a1 part2 中相同 (使用目标端口 of.OFPP_FLOOD)。但是，在核心路由器 (cores21) 中，您需要为所有 IP 流量指定特定端口。您可以按照自己的意愿执行此操作。但是，您可能会发现使用目标 IP 地址

和源 IP 地址以及数据包源自的交换机上的源端口来确定正确的目标端口和丢弃/允许操作是最简单的方法。

虽然所有授权主机都应该能够在它们自己和数据中心服务器之间进行通信，但我们需要保护网络免受不受信任的 Internet 的侵害。在此场景中，我们用 hnotrust1 表示外部主机，并具有以下要求：

1. 我们需要阻止从 hnotrust1 到 serv1 的所有 IP 流量，同时仍允许常规主机（h10、h20 等）与 hnotrust1 通信。
2. 为了阻止 Internet 发现我们的内部 IP 地址，我们希望阻止从 hnotrust1 到常规主机（h10、h20 等）和 serv1 的所有 ICMP 流量。

总结一下你的目标：

- 创建一个 Pox 控制器（按照 a1 第 2 部分），具有以下特点：所有节点都能够通信，除了
 - hnotrust1 无法向 h10、h20、h30 或 serv1 发送 ICMP 流量（但应该能够向主机发送 IP 流量）。
 - hnotrust1 无法向 serv1 发送任何 IP 流量。

任务：

1. 修改 a2part1controller.py 文件
2. 在多个 ssh 连接中，分别使用 `sudo ~/pox/pox.py misc.a2part1controller`，`sudo python3 ~/project-2/topos/part1.py` 来启动控制器和运行 mininet
3. 在进入的 CLI 中运行 pingall，除 hnotrust 之外的所有节点都应该能够发送和响应 ping。
4. 运行 `iperf hnotrust1 h10` 和 `iperf h10 serv1`。虽然这些命令中没有显示，但 hnotrust1 应该无法传输到 serv1，却能传输带其余节点。
5. 运行 `dpctl dump-flows`。这应该包含您插入到交换机中的所有规则。请将 3，4，5 中的输出截图附在实验报告中。

Part4：学习型路由器

学习目标：

- 编写一个程序来处理 ARP 请求并生成有效的 ARP 回复以建立无需静态地址映射的连接。
- 动态创建匹配 -> 操作规则，使用动态 L2 地址在多个不同的 L2 网络之间正确转发 L3 流量。
- 创建一个能够保持动态状态的 SDN 控制器程序，以响应网络中观察到的流量来配置多个交换机。
- 推理特定子网内的 L2 地址与用于网络间路由的 L3 地址之间的关系。

对于第 4 部分，我们将扩展第 3 部分的代码，以在 cores21 交换机之外实现更现实的 3 级路由器。a2part2controller.py 框架与第 1 部分非常相似，可以从第 3 部分复制一些功能开始。对于拓扑，我们再次提供了一个文件 (part2.py)。part1.py 和 part2.py 拓扑之间的区别在于，不再有静态 L3<->L2 地址映射加载到每个主机中，并且默认路由“h{N}0-eth0”更改为“via 10.0.{N}.1”，其中“10.0.{N}.1”是该特定子网的网关（即路由器）的

IP 地址。这有效地将网络从交换网络（主机发送到 MAC 地址）更改为路由网络（主机发送到 IP 地址，这可能需要 L2 网络之外的网关）。最小的 a2part1controller 将无法在此新拓扑上运行！

为了处理这个 L2<->L3 映射，cores核心 21 需要：

- 处理多个子网中的 ARP 流量（无需转发）；
- 在需要时生成有效的 ARP 回复；并且
- 跨链接域转发 IP 流量（这将需要更新 L2 标头）；

此外，此作业要求您的实现能够动态工作。您不能在启动时在 cores21 上安装静态路由。相反，您的路由器必须了解哪些端口和哪些 L2 地址对应于特定的 L3 地址，并动态地将适当的规则安装到 cores21 交换机中。可以通过处理收到的 ARP 消息的内容或网络上的其他流量（尽管处理 ARP 就足够了）来推断此信息。您可以在控制器中处理第 2 部分中的每个单独的 ARP 数据包（即不使用流规则），但大多数 IP 流量应该使用流规则来处理以提高效率。其他交换机（例如 s1）不需要修改，可以继续使用静态规则泛洪流量。

对于项目的这一部分，您应该丢弃路由器尚未获知适当目的地的数据包。这意味着必须丢弃一些数据包，而如果您的路由器有更多信息，这些数据包可以路由。请参阅扩展 1 以解决此缺陷。

您的实施仍应应用第 1 部分中的相同 L3 策略规则，特别是：

1. 我们需要阻止从 hnotrust1 子网到 serv1 的所有 IP 流量，同时仍允许常规主机（h10、h20 等）与 hnotrust1 通信。
2. 为了阻止 Internet 发现我们的内部 IP 地址，我们希望阻止从 hnotrust1 子网到常规主机（h10、h20 等）和 serv1 的所有 ICMP 流量。

Note: 将学习与 L3 策略结合起来在实践中并不是很安全，因为在现实世界中没有办法用纯 IP 来验证 L3 源地址。不要天真地采用本作业中使用的方法并将其应用于生产网络。

总而言之，您的完整实施应该：

- 处理多个子网中的 ARP 流量（无需转发）
- 在需要时生成有效的 ARP 回复
- 在 L2 网络内以及跨 L2 网络转发 IP 流量（这可能需要更新 L2 报头）
- 通过侦听 ARP 流量（或其他方法）动态了解网络的 L3 拓扑（哪些子网可通过哪些端口访问）
- 允许所有主机在获知目标地址后与服务器以及彼此之间进行双向通信
- 阻止 hnotrust1 向 serv1 发送 IP 流量
- 阻止 hnotrust1 向任何常规主机（h10、h20 等）或 serv1 发送 ICMP
- 允许常规主机（h10、h20 等）与 hnotrust1 之间的双向 IP 通信

任务：

1. 修改 a2part2controller.py
2. 在多个 ssh 连接中，分别使用 `sudo ~/pox/pox.py misc.a2part2controller`，`sudo python3 ~/project-2/topos/part2.py` 来启动控制器和运行 mininet
3. 在进入的 CLI 中运行 pingall，除 hnotrust 之外的所有节点都应该能够发送和响应 ping。请注意，当路由器了解路由时，某些 ping 会失败（为什么会这样？）。
4. 运行 `iperf hnotrust1 h10` 和 `iperf h10 serv1`。hnotrust 应该无法转移到 serv1，但应该能够转移到其他主机
5. 运行 `dpctl dump-flows`。这应该包含您插入到交换机中的所有规则。请将 3，4，5 中

的输出截图附在实验报告中。

注意事项

验收 DDL: 前两个Part 12.30 周二课上；后两个Part 1.8 周四课上

提交 DDL: 1 月 1 1 日 23: 59 之前发到助教邮箱。

提交物包括实验报告以及 part1-4 中修改的四个文件（高亮部分）。实验报告中包括 part1-4 的要求截图部分以及关键代码解释。